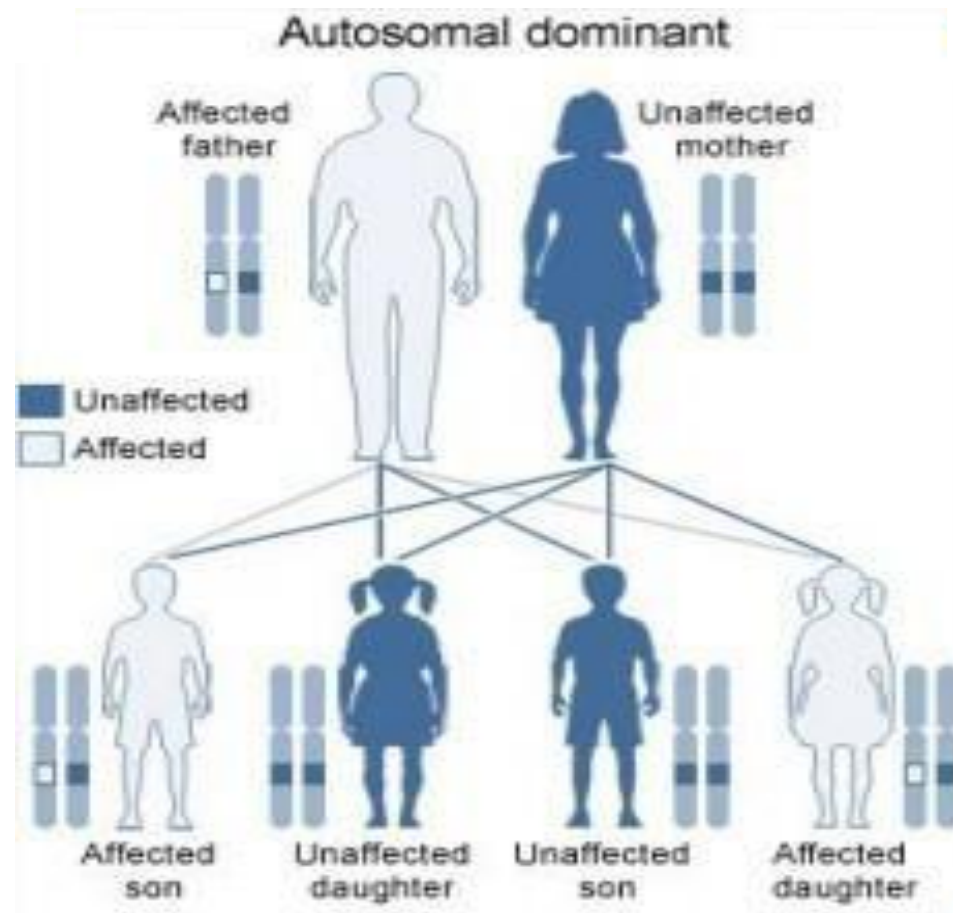


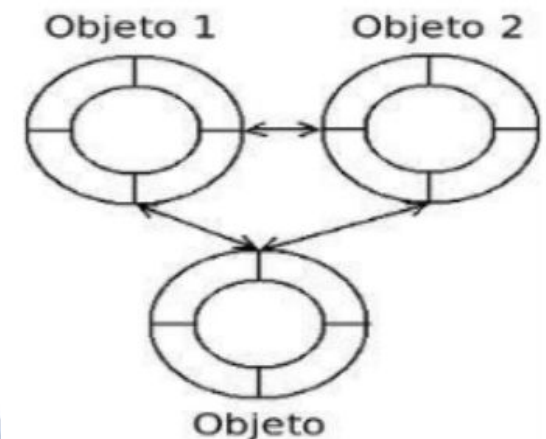
Orientação a Objetos – Relações, Herança e Polimorfismo



Relações

As **relações** descrevem como objetos ou classes se conectam entre si.

- Objetos não existem isolados:
 - São formados por outros objetos;
 - Objetos usam outros objetos;
 - Um programa OO possui vários objetos que interagem entre si;
 - Modelagem define quais objetos usamos em um programa e como eles se relacionam.



Relações

- Objetos possuem relacionamentos
 - Associação, Composição e Agregação.
 - Um objeto pode conter outros objetos;
 - Carro (motor, pneu, porta).
- Dependência
 - Objetos podem usar outros objetos;
 - Trem usa estrada de ferro.
- Herança
 - Uma subclasse e uma superclasse;
 - Leão é um animal.



Relações - Associação

- Associação representa uma relação estável entre duas classes, onde uma mantém referência direta à outra. Ambas as classes podem existir independentemente, mas uma conhece a outra e interage com ela.
- Exemplo: Um Professor está associado a um Departamento, mas ambos podem existir separadamente.
 - O Professor possui uma referência ao Departamento.



Relações - Associação

```
class Departamento:
    def __init__(self, nome):
        self.nome = nome

class Professor:
    def __init__(self, nome, departamento):
        self.nome = nome
        self.departamento = departamento # Associação com
Departamento

    def exibir_info(self):
        print(f"Professor: {self.nome}")
        print(f"Departamento: {self.departamento.nome}")

# Criando o departamento
dep_comp = Departamento("Computação")

# Criando o professor associado ao departamento
prof = Professor("Henrique", dep_comp)

# Exibindo informações
prof.exibir_info()
```

Relações - Composição

- É um tipo forte de associação em que uma classe "contém" outra como parte integrante de sua existência — ou seja, se o objeto "pai" for destruído, os objetos "filhos" também são;
- Um livro contém capítulos
 - Capítulo é parte essencial de livro;
 - Se não existir capítulo, não existe livro (capítulo não existe fora de livro).
- Linha com losângulo preenchido na classe “dominante”
 - Livro é composto de 1 ou mais capítulos.



Relações - Composição

```
class Capitulo:
```

```
    def __init__(self, titulo, numero):
```

```
        self.titulo = titulo
```

```
        self.numero = numero
```

```
    def exibir(self):
```

```
        print(f"Capítulo {self.numero}: {self.titulo}")
```

```
class Livro:
```

```
    def __init__(self, titulo):
```

```
        self.titulo = titulo
```

```
        self.capitulos = [] # Composição: o Livro contém os Capítulos
```

```
    def adicionar_capitulo(self, titulo):
```

```
        numero = len(self.capitulos) + 1
```

```
        cap = Capitulo(titulo, numero)
```

```
        self.capitulos.append(cap)
```

```
    def exibir_livro(self):
```

```
        print(f"Livro: {self.titulo}")
```

```
        for cap in self.capitulos:
```

```
            cap.exibir()
```

```
# Criando o livro com capítulos
```

```
livro = Livro("Python para Iniciantes")
```

```
livro.adicionar_capitulo("Introdução à  
Programação")
```

```
livro.adicionar_capitulo("Estruturas de Controle")
```

```
livro.adicionar_capitulo("Funções e Módulos")
```

```
# Exibindo conteúdo do livro
```

```
livro.exibir_livro()
```

Relações - Composição



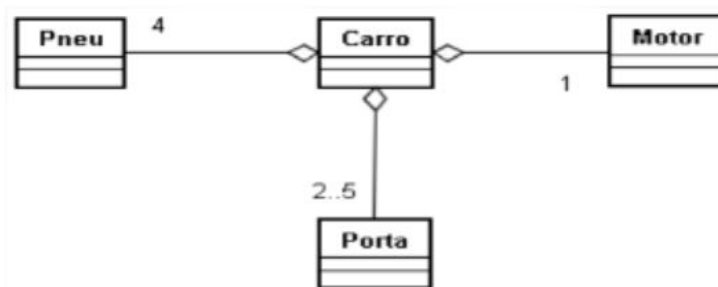
```
class Livro:
    def __init__(self, titulo):
        self.titulo = titulo
        self.capitulos = [ ]
```

```
class Capítulo:
    def __init__(self, titulo, numero, livro):
        self.titulo = titulo
        self.numero = numero
        self.livro = livro
```

Referência pode ou não
ser bidirecional. Capítulo
não precisa ter o atributo
livro

Relações - Agregação

- Agregação é uma associação mais fraca que a composição. Os objetos podem existir independentemente da classe agregadora;
- Carro contém Pneu, Motor e Porta
 - Não são partes essenciais do carro.
 - Retirando as portas um carro continua sendo um carro;
 - Pneus/Portas existem como objetos independentes.
- Linha com losângulo vazio na classe “dominante”.



Relações - Agregação

```
class Motor:  
    def __init__(self, tipo):  
        self.tipo = tipo
```

```
class Pneu:  
    def __init__(self, marca):  
        self.marca = marca
```

```
class Porta:  
    def __init__(self, lado):  
        self.lado = lado
```

```
class Carro:  
    def __init__(self, modelo, motor, pneus, portas):  
        self.modelo = modelo  
        self.motor = motor    # agregação  
        self.pneus = pneus    # lista de objetos Pneu  
        self.portas = portas  # lista de objetos Porta  
  
    def exibir_info(self):  
        print(f"Carro: {self.modelo}")  
        print(f"Motor: {self.motor.tipo}")  
  
        print("Pneus:")  
        for pneu in self.pneus:  
            print(f" - {pneu.marca}")  
  
        print("Portas:")  
        for porta in self.portas:  
            print(f" - {porta.lado}")
```

Relações - Agregação

```
# Criando as partes
motor_v8 = Motor("V8")
pneus = [Pneu("Michelin"), Pneu("Michelin"), Pneu("Michelin"),
Pneu("Michelin")]
portas = [
    Porta("Dianteira esquerda"),
    Porta("Dianteira direita"),
    Porta("Traseira esquerda"),
    Porta("Traseira direita")
]

# Criando o carro com agregação
carro = Carro("Mustang GT", motor_v8, pneus, portas)

# Exibindo informações
carro.exibir_info()
```

Relações – Dependência

- É a relação mais fraca entre classes. Ela ocorre quando uma classe usa outra de forma temporária, normalmente como parâmetro em métodos, ou em instâncias locais. Ou seja, uma classe depende da outra para executar alguma tarefa, mas não mantém uma relação contínua ou fixa com ela.



Relações – Dependência

```
class EstradaDeFerro:
    def __init__(self, nome):
        self.nome = nome

    def verificar_trilho(self):
        print(f"Trilho da estrada '{self.nome}' verificado e seguro para a viagem.")

class Trem:
    def __init__(self, modelo):
        self.modelo = modelo

    def usar_estrada_de_ferro(self, ferrovia):
        print(f"Trem modelo {self.modelo} solicitando uso da estrada de ferro.")
        ferrovia.verificar_trilho() # Uso temporário (dependência)
        print(f"Trem modelo {self.modelo} autorizado a seguir viagem.")

# Criando objetos com inicialização
ferrovia = EstradaDeFerro("Linha Sul")
trem = Trem("Locomotiva 3000")

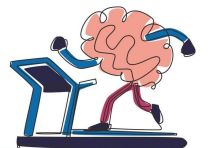
# O trem usa a estrada de ferro
trem.usar_estrada_de_ferro(ferrovia)
```

Relações

Relação	Descrição	Independência dos objetos	Grau de Acoplamento	Exemplo Python
Associação	Relação onde objetos se conhecem e colaboram, mas cada um tem ciclo de vida próprio.	Alta (objetos independentes)	Fraco	Professor e Departamento
Agregação	Relação "todo-parte" onde o todo contém partes criadas fora dele.	Alta (partes existem independentemente do todo)	Médio	Carro com Motor , Pneu , Porta
Composição	Relação "todo-parte" forte, onde o todo controla o ciclo de vida das partes.	Baixa (partes dependem do todo)	Forte	Livro e seus Capítulos
Dependência	Uso temporário de um objeto dentro de um método ou função de outro objeto.	Alta (uso temporário)	Muito fraco	Trem usando EstradaDeFerro no método

Exercício

- Implemente um sistema orientado a objetos em Python para representar uma biblioteca com as classes Biblioteca, Livro, Autor e Usuário, aplicando os conceitos de associação, agregação, composição e dependência. A classe Autor deve armazenar nome e nacionalidade; a classe Livro deve conter título, ano e uma referência a um Autor (agregação). A Biblioteca deve possuir um nome e ser responsável por criar e armazenar os livros internamente (composição). A classe Usuario deve conter o nome e manter uma referência à Biblioteca à qual está associado (associação), além de possuir um método para pegar um livro emprestado, usando temporariamente o livro sem armazená-lo (dependência).



Exercício

- Implemente um **sistema de biblioteca** em Python utilizando **Orientação a Objetos**. O sistema deve permitir gerenciar livros, autores e usuários, possibilitando empréstimos de forma simplificada.

Requisitos:

Classes principais:

Autor: armazena nome e nacionalidade.

Livro: armazena título, ano e referência a um Autor.

Biblioteca: armazena nome e mantém internamente os livros; deve permitir adicionar e listar livros.

Usuário: armazena nome e mantém referência à biblioteca; deve permitir pegar livros emprestados temporariamente.

Funcionalidades do sistema:

Listar livros disponíveis.

Adicionar livros à biblioteca.

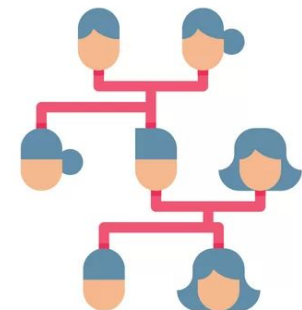
Permitir empréstimo de livros pelo usuário.

Encerrar o sistema.

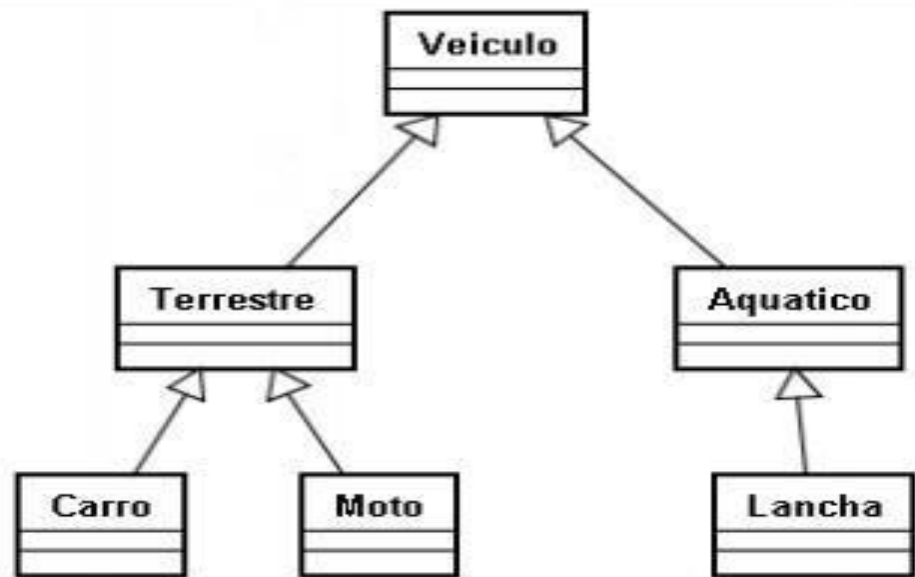


Herança

- Herança é um princípio da programação orientada a objetos (POO) que permite criar uma nova classe (subclasse/filha) a partir de uma classe existente (superclasse/mãe), reaproveitando código e facilitando a organização;
- Como funciona:
 - A subclasse herda atributos e métodos da superclasse;
 - Pode adicionar novos atributos/métodos;
 - Pode sobrescrever (ou redefinir) métodos da superclasse para modificar o comportamento.



Herança



Herança

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        print(f"Nome: {self.nome}, Idade: {self.idade}")

class Aluno(Pessoa): # Herda de Pessoa
    def __init__(self, nome, idade, matricula):
        super().__init__(nome, idade) # Reaproveita o construtor da superclasse
        self.matricula = matricula    # Novo atributo da subclasse

    def estudar(self): # Novo método exclusivo da subclasse
        print(f"{self.nome} está estudando. Matrícula: {self.matricula}")

p = Pessoa("Ana", 30)
p.apresentar() # Nome: Ana, Idade: 30

a = Aluno("Bruno", 20, "2025A")
a.apresentar() # Método herdado da superclasse
a.estudar()    # Método da subclasse
```

Herança

- `super()` é uma função embutida em Python que retorna uma referência à superclasse (ou classe pai) da classe atual. É usada principalmente para chamar métodos da superclasse, especialmente dentro de construtores (`__init__`) e métodos sobrescritos;
- Por que usar `super()`?
 - Para evitar repetir código que já está na superclasse;
 - Para garantir que a lógica da classe mãe seja preservada;
 - Para tornar o código mais flexível e seguro, principalmente em herança múltipla.



Herança

```
class Pessoa:
```

```
    def __init__(self, nome, idade):  
        self.nome = nome  
        self.idade = idade
```

```
class Aluno(Pessoa):
```

```
    def __init__(self, nome, idade, matricula):  
        super().__init__(nome, idade)  
        self.matricula = matricula
```

```
class Aluno(Pessoa):
```

```
    def __init__(self, nome, idade, matricula):  
        self.nome = nome  
        self.idade = idade  
        self.matricula = matricula
```

```
class Pessoa:
```

```
    def falar(self):  
        print("Pessoa está falando.")
```

```
class Professor(Pessoa):
```

```
    def falar(self):  
        super().falar() # Chama o método da classe mãe  
        print("Professor está explicando.")
```

Herança

1. Superclasse

2. Classe Base

3. Classe pai/mãe

4. Classe ancestral

1. Subclasse

2. Classe Derivada

3. Classe filha

4. Classe descendente

**sinônimo
de amor
é AMAR.**

Herança

Superclasse

É uma classe a ser herdada por outra

É uma classe geral

Generalização

Subclasse

É uma classe que herda de outra

É uma classe específica

Especialização

Classe ancestral

É uma classe que está mais acima

Em relação a outras em uma hierarquia de classes

Generalização

Classe descendente

É uma classe que está mais abaixo

Em relação a outras em uma hierarquia de classes

Especialização

Herança – Redefinição de Métodos

- Sobrescrita de métodos (override) ocorre quando uma subclasse define um método com o mesmo nome da superclasse, substituindo seu comportamento. Em Python, isso ocorre automaticamente se a subclasse declarar um método com o mesmo nome. A sobrescrita só acontece se o nome do método for o mesmo e a quantidade de parâmetros for compatível;
- Por que usar sobrescrita?
 - Para alterar ou especializar o comportamento da superclasse;
 - Para criar métodos mais adequados ao contexto da subclasse;
 - Para aplicar o polimorfismo — objetos diferentes respondem ao mesmo método com comportamentos distintos.

Herança – Redefinição de Métodos

```
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def emitir_som(self):
        print(f"{self.nome} faz um som genérico.")

class Cachorro(Animal):
    def emitir_som(self): # Sobrescrevendo método da superclasse
        print(f"{self.nome} diz: Au au!")

class Gato(Animal):
    def emitir_som(self):
        print(f"{self.nome} diz: Miau!")

# Criando objetos
animal = Animal("Animal genérico")
cachorro = Cachorro("Rex")
gato = Gato("Mimi")

animal.emitir_som() # Animal genérico faz um som genérico.
cachorro.emitir_som() # Rex diz: Au au!
gato.emitir_som() # Mimi diz: Miau!
```

Herança – Redefinição de Métodos

- Às vezes você quer sobrescrever um método sem perder o comportamento original. Use `super()` para chamar o método da superclasse:

```
class Pessoa:  
    def apresentar(self):  
        print("Olá, sou uma pessoa.")
```

```
class Professor(Pessoa):  
    def apresentar(self): # Sobrescreve o método  
        print("Olá, sou um professor.")
```

```
class Professor(Pessoa):  
    def apresentar(self):  
        super().apresentar() # Chama o método da superclasse  
        print("E também sou um professor.")
```

Herança – Sobrecarga de Métodos

- Sobrecarga de métodos é um conceito onde uma classe pode ter vários métodos com o mesmo nome, mas com assinaturas diferentes (isto é, o número ou tipo de parâmetros são diferentes). Isso é comum em linguagens como Java, mas em Python o conceito funciona de maneira diferente, pois o Python não suporta sobrecarga de métodos diretamente como outras linguagens;
- Em Python, a sobrecarga de métodos não é suportada de forma nativa como em outras linguagens (Java, C++), mas podemos simular a sobrecarga usando diferentes técnicas:
 - Argumentos variáveis (*args);
 - Verificação de tipos;
 - Parâmetros padrão.



Herança – Sobrecarga de Métodos

- **Usando argumentos variáveis (*args):** Uma forma de simular a sobrecarga é usar *args (para parâmetros posicionais), que permite que um método aceite um número variável de argumentos.

```
class Calculadora:
    def somar(self, *args):
        return sum(args)

# Testando a sobrecarga
calc = Calculadora()
print(calc.somar(1, 2))      # 3
print(calc.somar(1, 2, 3, 4)) # 10
```


Herança – Sobrecarga de Métodos

- **Verificando o tipo de argumento:** Outra abordagem é verificar o tipo dos argumentos passados e, com isso, alterar o comportamento do método. Isso simula a sobrecarga, pois o comportamento do método será adaptado conforme o tipo de argumento.

```
class Operacoes:
    def somar(self, a, b):
        if isinstance(a, int) and isinstance(b, int): # Ambos inteiros
            return a + b
        elif isinstance(a, str) and isinstance(b, str): # Ambos strings
            return a + " " + b
        else:
            return "Operação inválida: os argumentos devem ser ambos inteiros ou ambos strings."
```

```
op = Operacoes()
```

```
print(op.somar(3, 4))          # 7
print(op.somar("Olá", "Mundo")) # "Olá Mundo"
print(op.somar(3, "Mundo"))    # Operação inválida...
```


Herança – Sobrecarga de Métodos

- `isinstance()` é uma função embutida do Python usada para verificar se um objeto é instância de uma classe (ou de uma tupla de classes);
- Sintaxe: `isinstance(objeto, classe_ou_tupla_de_classes)`

```
class Pessoa:  
    pass
```

```
class Aluno(Pessoa):  
    pass
```

```
p = Pessoa()  
a = Aluno()
```

```
print(isinstance(p, Pessoa)) # True  
print(isinstance(a, Pessoa)) # True (Aluno herda de  
Pessoa)  
print(isinstance(a, Aluno)) # True  
print(isinstance(p, Aluno)) # False
```

```
print(isinstance(10, int))      # True  
print(isinstance("texto", str)) # True  
print(isinstance(3.14, float)) #  
True  
print(isinstance(10, str))      # False
```

Herança – Sobrecarga de Métodos

- **Parâmetros padrão:** São parâmetros que têm um valor predefinido e podem ser omitidos ao chamar a função ou método;
- **Sintaxe básica:** `def funcao(param1, param2=valor_padrao)`

```
class Calculadora:
```

```
    def operar(self, a=0, b=0, operacao="+"):
```

```
        if operacao == "+":
```

```
            return a + b
```

```
        elif operacao == "-":
```

```
            return a - b
```

```
        elif operacao == "*":
```

```
            return a * b
```

```
        elif operacao == "/" and b != 0:
```

```
            return a / b
```

```
        else:
```

```
            return "Operação inválida ou divisão
```

```
por zero"
```

```
calc = Calculadora()
```

```
print(calc.operar()) # 0 + 0 = 0
```

```
print(calc.operar(5)) # 5 + 0 = 5
```

```
print(calc.operar(5, 3)) # 5 + 3 = 8
```

```
print(calc.operar(5, 3, "-")) # 5 - 3 = 2
```

```
print(calc.operar(5, 3, "*")) # 5 * 3 = 15
```

```
print(calc.operar(5, 0, "/")) # divisão por zero
```

```
print(calc.operar(5, 2, "/")) # 5 / 2 = 2.5
```

Herança Múltipla

- Python suporta herança múltipla, ou seja, uma classe pode herdar de duas ou mais classes ao mesmo tempo.

```
class A:  
    def metodo_a(self):  
        print("Método da classe A")
```

```
class B:  
    def metodo_b(self):  
        print("Método da classe B")
```

```
class C(A, B): # Herda de A e B  
    def metodo_c(self):  
        print("Método da classe C")
```

```
obj = C()  
obj.metodo_a() # Método da classe A  
obj.metodo_b() # Método da classe B  
obj.metodo_c() # Método da classe C
```

Herança Múltipla

- Quando duas superclasses têm o mesmo método, o Python resolve isso usando o MRO (Method Resolution Order) — ou seja, a ordem das classes entre parênteses importa.

```
class A:  
    def falar(self):  
        print("Classe A")
```

```
class B:  
    def falar(self):  
        print("Classe B")
```

```
class C(A, B): # Ordem: A depois B  
    pass
```

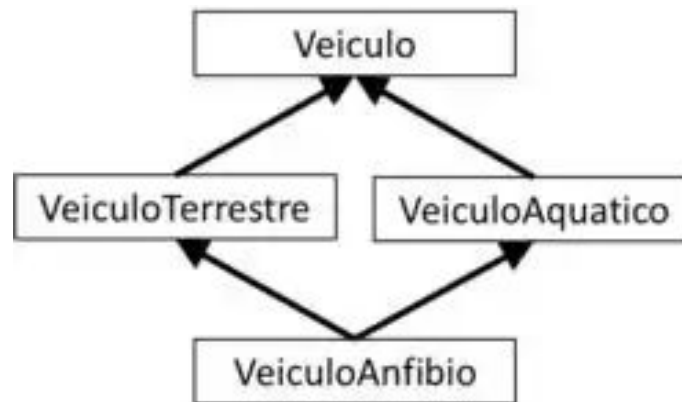
```
c = C()  
c.falar() # Saída: Classe A
```

```
class C(B, A): # Agora B vem antes  
    pass
```

```
c = C()  
c.falar() # Saída: Classe B
```

Herança Múltipla

- Cuidados com herança múltipla:
 - Pode causar ambiguidade se mal usada;
 - Prefira composição se não for necessário herdar comportamentos complexos;
 - Use herança múltipla quando as classes realmente representam papéis distintos e reutilizáveis.



Herança Múltipla

```
class GuerreiroArmado:
```

```
    def __init__(self, armadura):  
        self.armadura = armadura
```

```
    def vestir_armadura(self):  
        print(f"Vestindo a armadura de {self.armadura}!")
```

```
    def atacar(self):  
        print("Atacando com a força da armadura!")
```

```
class PoderCosmico:
```

```
    def __init__(self, nivel_cosmico):  
        self.nivel_cosmico = nivel_cosmico
```

```
    def liberar_cosmo(self):  
        print(f"Liberando cosmo no nível {self.nivel_cosmico}!")
```

```
    def atacar(self):  
        print("Atacando com o poder do cosmo!")
```

Herança Múltipla

Herança múltipla

```
class Cavaleiro(GuerreiroArmado, PoderCosmico):
```

```
    def __init__(self, nome, armadura, nivel_cosmico):
```

```
        GuerreiroArmado.__init__(self, armadura)
```

```
        PoderCosmico.__init__(self, nivel_cosmico)
```

```
        self.nome = nome
```

```
    def apresentar(self):
```

```
        print(f"Eu sou {self.nome}, Cavaleiro de {self.armadura}!")
```

Podemos escolher sobrescrever o método atacar ou usar o da superclasse

Aqui, vamos usar o método atacar da classe GuerreiroArmado (primeira da herança)

Teste

```
seiya = Cavaleiro("Seiya", "Pégaso", 9000)
```

```
seiya.apresentar()
```

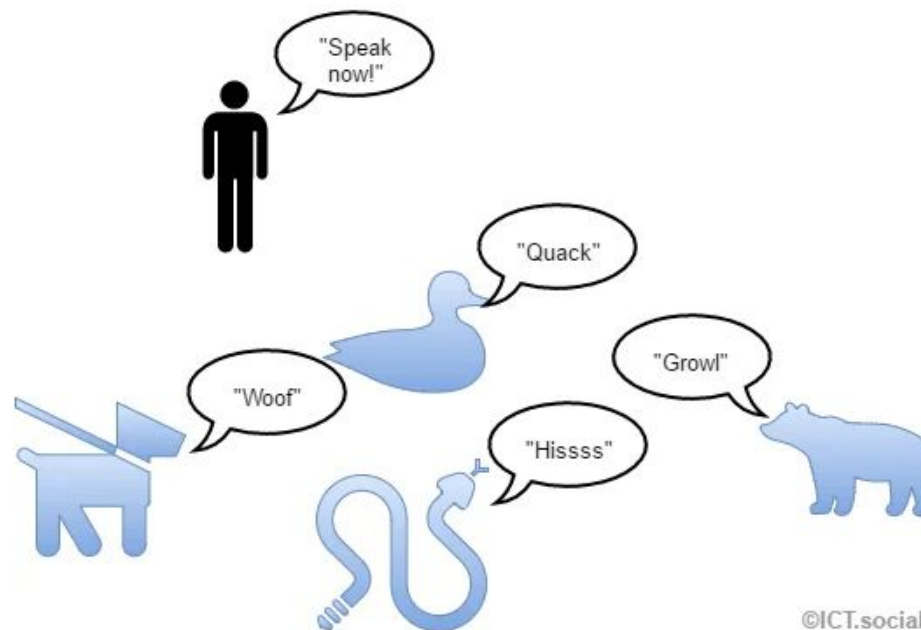
```
seiya.vestir_armadura()
```

```
seiya.liberar_cosmo()
```

```
seiya.atacar() # Chama o método atacar da classe GuerreiroArmado porque está primeiro na herança
```


Polimorfismo

- Polimorfismo significa “muitas formas”. É a capacidade de diferentes classes responderem a uma mesma mensagem (ou chamada de método) de formas específicas. Ou seja, objetos de classes diferentes podem ser usados através da mesma interface (mesmo nome de método), mas com comportamentos diferentes.



Polimorfismo

```
class Personagem:  
    def atacar(self):  
        print("Personagem genérico atacando!")
```

```
class Goku(Personagem):  
    def atacar(self):  
        print("Goku lança um Kamehameha!")
```

```
class Vegeta(Personagem):  
    def atacar(self):  
        print("Vegeta dispara um Final Flash!")
```

```
class Freeza(Personagem):  
    def atacar(self):  
        print("Freeza usa o Death Beam!")
```

```
def iniciar_batalha(personagem):  
    personagem.atacar()
```

Testando polimorfismo

```
goku = Goku()  
vegeta = Vegeta()  
freeza = Freeza()
```

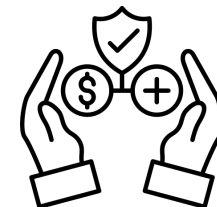
```
iniciar_batalha(goku)    # Goku lança  
um Kamehameha!
```

```
iniciar_batalha(vegeta)  # Vegeta  
dispara um Final Flash!
```

```
iniciar_batalha(freeza)  # Freeza usa o  
Death Beam!
```

Polimorfismo

- Benefícios do polimorfismo:
 - Flexibilidade: permite usar objetos de diferentes classes de forma intercambiável;
 - Reuso de código: funções/métodos funcionam para várias classes sem alteração;
 - Desacoplamento: reduz dependência do código em implementações específicas;
 - Facilidade de manutenção: mudanças nas subclasses não afetam o código que as usa;
 - Interface uniforme: objetos diferentes respondem aos mesmos métodos, simplificando o código.



Exercício

- Crie um programa em Python utilizando herança para representar um sistema simples de funcionários. Implemente uma classe base chamada `Funcionario`, com os atributos `nome` (`String`) e `salarioBase` (`double`), além de um método `calcularSalario()` que retorna o salário base e um método `exibirDados()` que imprime o nome e o salário. Em seguida, crie uma subclasse chamada `FuncionarioComissionado`, que herda de `Funcionario` e possui o atributo adicional `comissao` (`double`). Essa subclasse deve sobrescrever o método `calcularSalario()` para retornar a soma do salário base com a comissão, e também sobrescrever `exibirDados()` para incluir a comissão nas informações exibidas. Por fim, instancie um objeto de cada classe e utilize os métodos definidos para mostrar os dados dos funcionários.



FIM

